

Test-Driven Development

an introduction to TDD and unit testing

what it is, why it matters, and how to get started



testing is a vague word in software
development; there are many different kinds

beta-testing • performance-testing • stress-testing •
integration-testing • regression-testing • usability-testing

this guide focuses exclusively on unit-testing:
testing we do as programmers, not as end-users



testing the **individual** units of our
code, like **classes** and **methods**

unit-testing

proving that if we pass these
particular values into that
particular **method**, we will get
this **specific result** back

testing code **as it is being**
written; even before a
user interface or most
classes exist



automated unit-tests



manually test your application



write logic that tests our code

you don't want to manually call testing logic and check results
against a massive sheet of outputs every single time

test code lives alongside application code, and considered equally important



writing unit-tests is done very simply and
requires minimal code; easy enough to run
ten, twenty, or a hundred times a day

having automated unit tests allows us to easily validate
that any change we make; whether tomorrow or six
months later; didn't break something we did earlier

with Test-Driven Development we're unit-
testing, but that's not all we are doing! To do TDD,
we have to take that idea one step further



1

write normal
application code

2

write automated
unit tests

```
public double calculatePrice(String itemId, int quantity) {  
    // whatever  
}
```



NOT TDD

```
calculatePrice("146", 3); // is result 120.60 ?  
calculatePrice("109", 8); // is result 8.08 ?
```



1

write automated
unit tests

2

write normal
application code

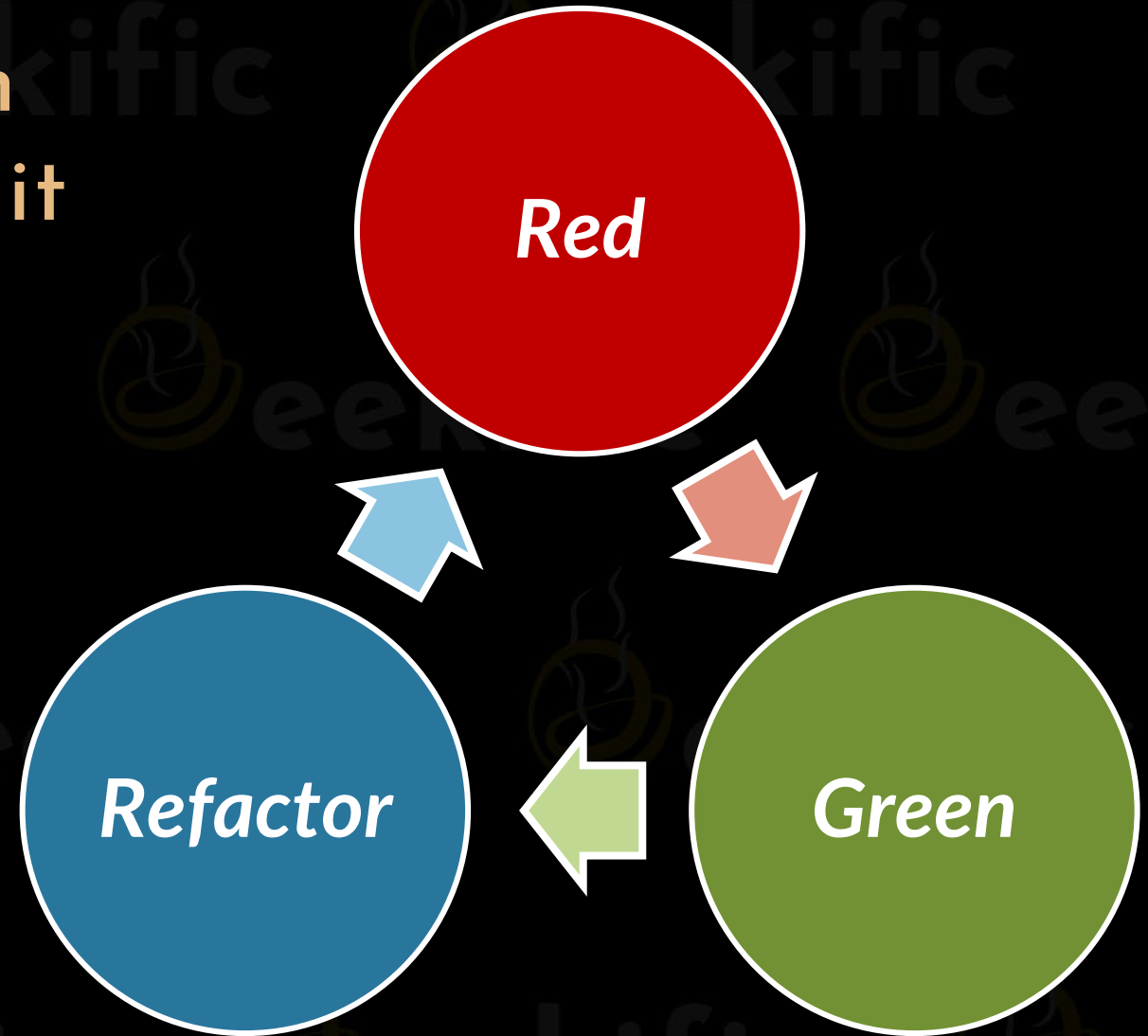
write tests that attempt to instantiate an
object which doesn't have a class yet

write tests that call a method that doesn't
exist and we will run those tests



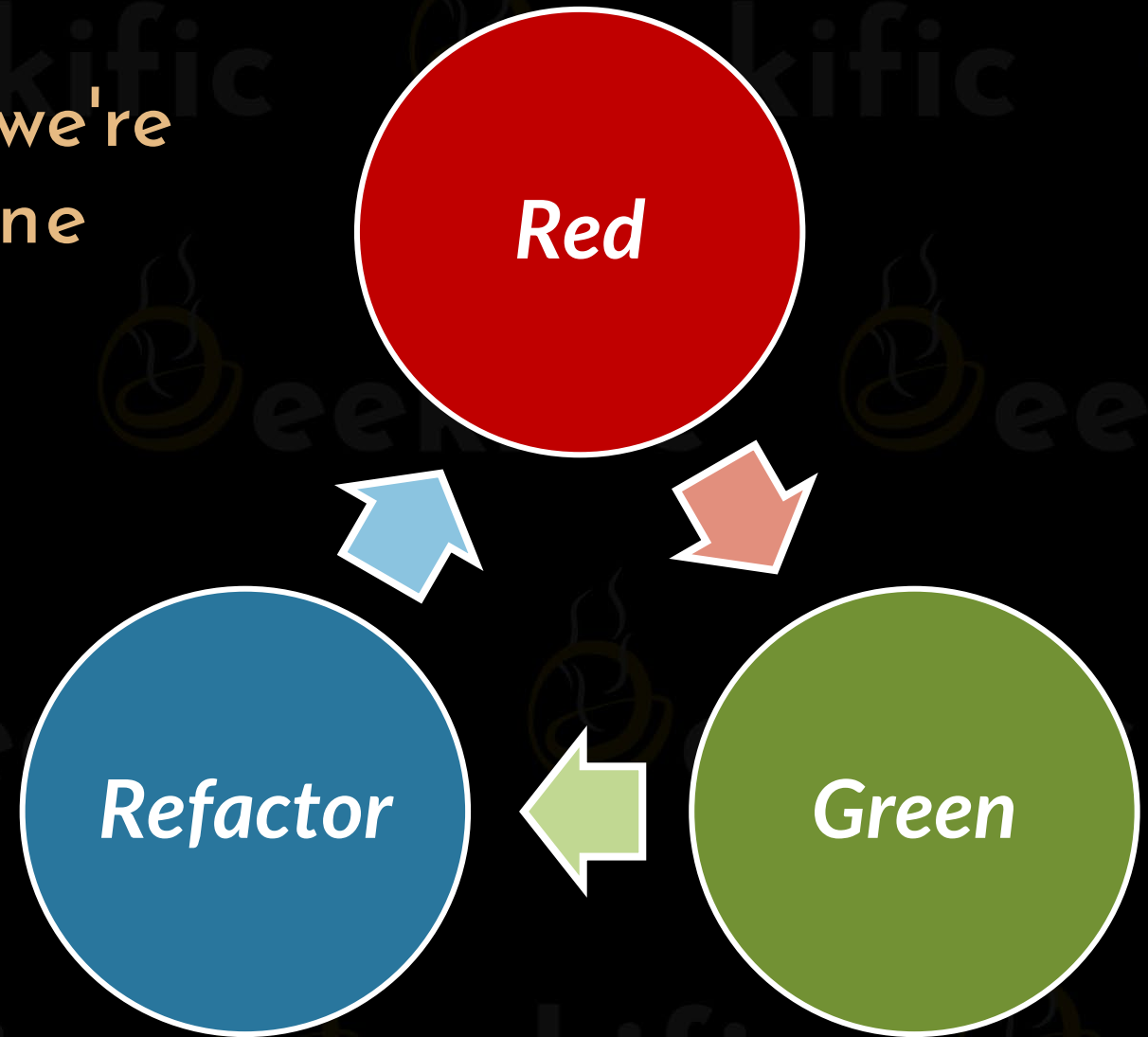
the first time we run
our unit-test we want it
to fail

then we write the
minimum necessary
code just to pass that
specific test



all the speculative code that we're
trying to come up with is gone

we just need pass this one
test, it gives us focus and
this is what we mean by
Test-Driven
Development



not just development that uses unit tests, it is a process and technique that **focuses on the priority and position given to those unit tests**

shifts the idea of **testing** from an afterthought, to something that will **drives everything else we do**

Test-Driven Development



is not about writing all your unit tests and then start writing your application logic; **it consists of very small repeated steps**

isn't here to replace other kinds of testing, but is **worthwhile for 80-90% of your code**



pause

write a test

verify it fails

pass the test



it can't be applied on everything

it doesn't fix all our testing problems

just like OOP, design patterns, and agile – these are excellent skills to have for your development ability and career



why use frameworks?

you could write a brand-new class with if/else conditions to test your logic but that approach is monotonous and requires duplicating code several times

unit-testing frameworks like JUnit, NUnit, and unittest provide proven, reusable tools to automate this process



```
Market myMarket = new Market();  
double result = myMarket.calculatePrice("146", 3);  
if (result == 120.60) {  
    System.out.println("test passed!");  
} else {  
    System.out.println("test failed");  
}
```

Unit-Testing Frameworks



developed by Kent
Beck in the 1990s

SUnit

JUnit 

 nunit


unittest



xUnit

xUnit is the most widely used style of unit testing

xUnit frameworks are by far the most common

if a unit testing framework is based on these original ideas, it is considered an xUnit testing framework

JUnit

NUnit

OCUnit

CppUnit



xUnit

learning to unit-test in one language transfers to others
– most frameworks share the same xUnit ideas

not all testing frameworks follow the xUnit style
– there are competing approaches



JUnit 5

unit-testing is extremely common in the Java world

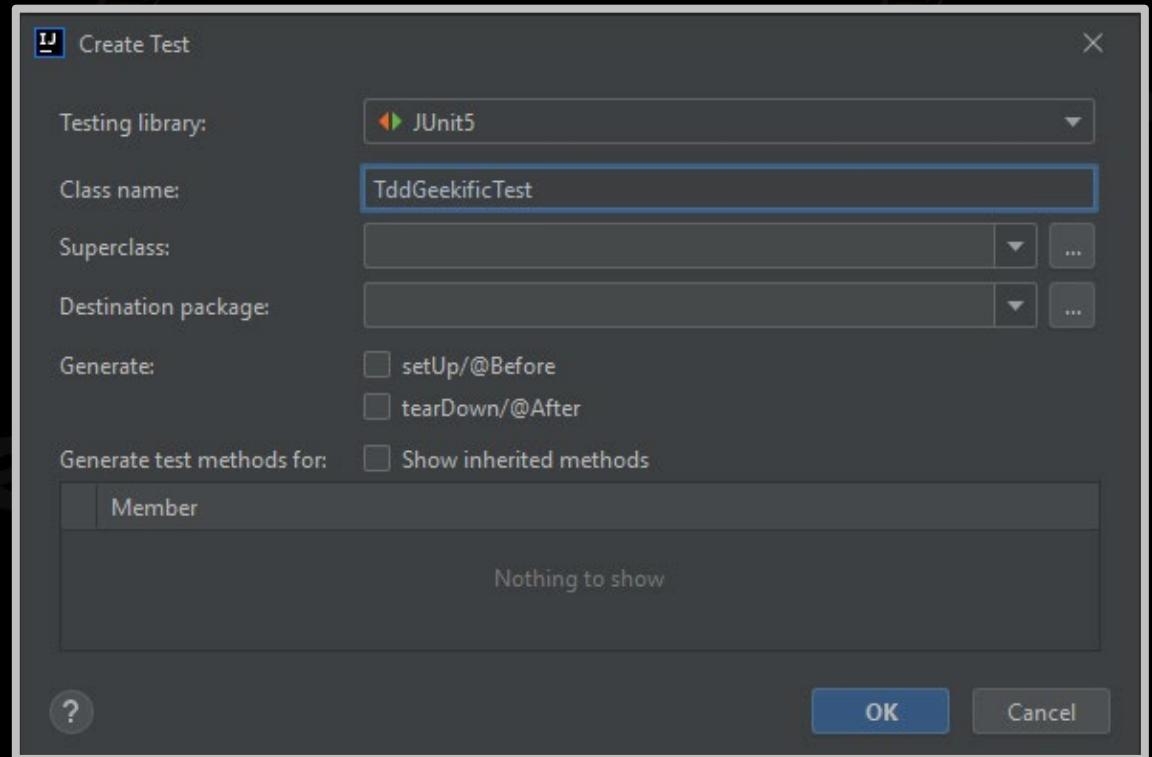
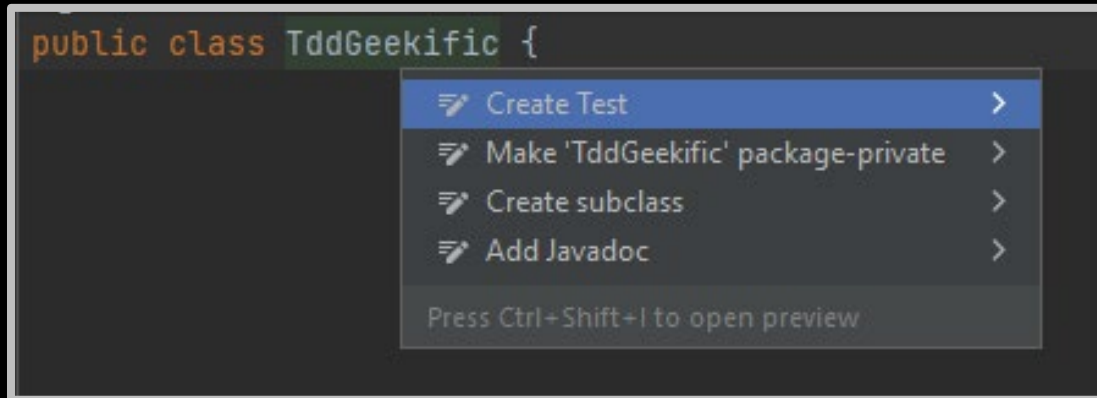
JUnit was the first framework to adopt the SUnit ideas

the first books on TDD were Java focused

most xUnit frameworks are really based on JUnit
rather than the official SUnit ancestor



most IDEs have built-in support of the most common unit-testing framework for the language you are using



unit testing proves that individual pieces of code work correctly
by passing specific values and checking specific results

TDD means writing a failing test first,
then writing the minimum code to pass it

the Red-Green-Refactor cycle keeps
development focused, one small step at a time

